In [1]:

```
# MantisV2 improvement: learned token pooling (idea 2) + cross-channel mi
xer (idea 1) on the frozen encoder
# Padmanabh Khunt (26100085), AI for Time Series group project
!pip install -q mantis-tsfm
import os, time, json, zipfile, subprocess, requests
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from mantis.architecture import MantisV2
from mantis.trainer import MantisTrainer

device = 'cuda'
DATA, OUT = '/tmp/data', '/kaggle/working'
os.makedirs(DATA, exist_ok=True)
net = MantisV2(device=device, return_transf_layer=2, output_token='combin
ed').from_pretrained('paris-noah/MantisV2')
net.return_transf_layer, net.output_token = 2, 'combined'
net.eval()
print(torch.cuda.get_device_name(0))
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Pleas
e set a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
Tesla T4
```

In [2]:

```
# Data loaders (same as the reproduction notebook)
def load_ts_archive(name):
    folder = f'{DATA}/{name}'
    if not os.path.exists(folder):
        subprocess.run(['wget', '-q', '-O', f'{DATA}/{name}.zip',
                        f'https://www.timeseriesclassification.com/aeon-t
oolkit/{name}.zip'], check=True)
        with zipfile.ZipFile(f'{DATA}/{name}.zip') as z:
            z.extractall(folder)
    out = {}
    for split in ['TRAIN', 'TEST']:
        path = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts'][0]
        X, y, in_data = [], [], False
        with open(path, encoding='utf-8', errors='ignore') as fh:
            for line in fh:
                line = line.strip()
                if not line or line.startswith('#'):
                    continue
                if line.lower().startswith('@data'):
                    in_data = True
                    continue
                if in_data:
                    *channels, label = line.split(':')
                    X.append([[float(v) if v != '?' else np.nan for v in
ch.split(',')] for ch in channels])
                    y.append(label)
        out[split.lower()] = (np.asarray(X, dtype=np.float32), np.asarray
(y))
    return out

TFC = {'HAR': 19930244, 'Epilepsy': 19930199}

def load_tfc(name):
    folder = f'{DATA}/{name}'
    os.makedirs(folder, exist_ok=True)
    files = requests.get(f'https://api.figshare.com/v2/articles/{TFC[nam
e]}', timeout=30).json()['files']
    out = {}
    for split in ['train', 'test']:
```

```
        path = f'{folder}/{split}.pt'
        if not os.path.exists(path):
            url = [f['download_url'] for f in files if f['name'] == f'{sp
lit}.pt'][0]
            with open(path, 'wb') as fh:
                fh.write(requests.get(url, timeout=600).content)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        if X.ndim == 2:
            X = X[:, None, :]
        out[split] = (X, y)
    return out
```

In [3]:

```python
# Frozen encoder -> all tokens of all 6 transformer layers
@torch.no_grad()
def tokens(X, bs=512):
    """Returns (N, C, 6 layers, 33 tokens, 256) in float16. Tokens 0-31 a
re patches, token 32 is the cls token."""
    X = np.nan_to_num(np.asarray(X, dtype=np.float32))
    N, C, _ = X.shape
    X = F.interpolate(torch.tensor(X), size=512, mode='linear', align_cor
ners=False).reshape(N * C, 1, 512)
    out = []
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(X[i:i + bs].to(device))
        cls = net.transf_unit.cls_token.expand(h.shape[0], 1, -1)
        h = torch.cat([h, cls], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            layers.append(h[:, 0])
        out.append(torch.stack(layers, 1).half().cpu())
    return torch.cat(out).reshape(N, C, 6, 33, 256)

def fixed_features(T):
    """The paper's features: layer 2, cls token + mean of patch tokens, c
hannels concatenated."""
    t = T[:, :, 2].float()
    return torch.cat([t[:, :, 32], t[:, :, :32].mean(2)], -1).reshape(len
(T), -1).numpy()

# sanity check: our token extraction must give the official MantisV2 feat
ures
Xc = np.random.randn(8, 2, 300).astype(np.float32)
official = MantisTrainer(device=device, network=net).transform(
    F.interpolate(torch.tensor(Xc), size=512, mode='linear', align_corner
s=False).numpy())
print('max difference to official features:', np.abs(fixed_features(token
s(Xc)) - official).max())
```

```
max difference to official features: 0.003446579
```

In [4]:

```
# The new head on top of the frozen encoder
class Head(nn.Module):
    """pool='fixed': paper aggregation (layer 2, cls + mean token). pool
='learned': softmax-weighted mix of the
    6 layers + attention pooling over the 33 tokens (idea 2).
    chan='concat': channels concatenated as in the paper. chan='mixer': t
ransformer across channels (idea 1)."""
    def __init__(self, C, K, pool, chan, d=128, heads=4, drop=0.2):
        super().__init__()
        self.pool, self.chan = pool, chan
        if pool == 'fixed':
            self.proj = nn.Sequential(nn.LayerNorm(512), nn.Linear(512,
d))
        else:
            self.layer_w = nn.Parameter(torch.zeros(6))
            self.norm = nn.LayerNorm(256)
            self.query = nn.Parameter(torch.randn(heads, 256) * 0.02)
            self.proj = nn.Sequential(nn.LayerNorm(heads * 256), nn.Linea
r(heads * 256, d))
        if chan == 'concat':
            self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d,
K))
        else:
            self.chan_emb = nn.Parameter(torch.randn(1, C, d) * 0.02)
            self.cls = nn.Parameter(torch.zeros(1, 1, d))
            layer = nn.TransformerEncoderLayer(d, 4, 2 * d, drop, batch_f
irst=True, norm_first=True)
            self.mixer = nn.TransformerEncoder(layer, 1, enable_nested_te
nsor=False)
            self.out = nn.Sequential(nn.LayerNorm(d), nn.Dropout(drop), n
n.Linear(d, K))

    def forward(self, T):
        T = T.float()
        if self.pool == 'fixed':
            t = T[:, :, 2]
            z = torch.cat([t[:, :, 32], t[:, :, :32].mean(2)], -1)
        else:
            w = torch.softmax(self.layer_w, 0)
            t = (self.norm(T) * w[:, None, None]).sum(2)      # weighted
```

```
mix of the 6 layers: (B, C, 33, 256)
            a = torch.softmax(t @ self.query.T / 16, 2)       # attention
over the 33 tokens: (B, C, 33, heads)
            z = (a.transpose(2, 3) @ t).flatten(2)            # (B, C, he
ads * 256)
        z = self.proj(z)                                      # (B, C, d)
        if self.chan == 'concat':
            return self.out(z.flatten(1))
        z = torch.cat([self.cls.expand(len(z), -1, -1), z + self.chan_em
b], 1)
        return self.out(self.mixer(z)[:, 0])
```

In [5]:

```
# Training (20% of the train split held out for early stopping) and the l
ogistic regression baseline
def train_head(Ttr, ytr, Tte, yte, C, K, pool, chan, seed, epochs=100, pa
tience=20, bs=64):
    torch.manual_seed(seed)
    np.random.seed(seed)
    idx_tr, idx_va = train_test_split(np.arange(len(ytr)), test_size=0.2,
stratify=ytr, random_state=seed)
    model = Head(C, K, pool, chan).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.0
5)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    ytr_t = torch.tensor(ytr, dtype=torch.long)

    def predict(T):
        model.eval()
        preds = []
        with torch.no_grad():
            for i in range(0, len(T), 512):
                preds.append(model(T[i:i + 512].to(device)).argmax(1).cpu
())
        return torch.cat(preds).numpy()

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(idx_tr)
        for i in range(0, len(perm), bs):
            b = torch.tensor(perm[i:i + bs])
            loss = F.cross_entropy(model(Ttr[b].to(device)), ytr_t[b].to
(device), label_smoothing=0.1)
            opt.zero_grad()
            loss.backward()
            opt.step()
        sched.step()
        va = np.mean(predict(Ttr[torch.tensor(idx_va)]) == ytr[idx_va])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.clone() for k, v in model.state_dict().ite
ms()}
```

```
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    n_params = sum(p.numel() for p in model.parameters())
    return float(np.mean(predict(Tte) == yte)), ep + 1, n_params

def logreg(Ztr, ytr, Zte, yte):
    s = StandardScaler().fit(Ztr)
    clf = LogisticRegression(max_iter=500).fit(s.transform(Ztr), ytr)
    return float(np.mean(clf.predict(s.transform(Zte)) == yte))
```

In [6]:

```
# Mixer v2: fixes for mixer v1 (too big, overfits on small data)
# 1) residual + gate that starts at 0: the model starts as the plain conc
at head and only uses mixing if it helps
# 2) smaller: d=64, 2 heads, one attention layer, no feed-forward block
# 3) channel dropout: whole channels are blanked at random while trainin
g, so the model cannot rely on one channel
emb = MantisTrainer(device=device, network=net)

def fixed(X):
    """Paper features kept per channel: (N, C, 512) in float16."""
    X = np.nan_to_num(np.asarray(X, dtype=np.float32))
    N, C, _ = X.shape
    X = F.interpolate(torch.tensor(X), size=512, mode='linear', align_cor
ners=False).numpy()
    return emb.transform(X).reshape(N, C, 512).astype(np.float16)

class MixerV2(nn.Module):
    def __init__(self, C, K, mix, d=64, drop=0.3, chan_drop=0.1):
        super().__init__()
        self.mix, self.chan_drop = mix, chan_drop
        self.proj = nn.Sequential(nn.LayerNorm(512), nn.Linear(512, d))
        if mix:
            self.chan_emb = nn.Parameter(torch.zeros(1, C, d))
            self.norm = nn.LayerNorm(d)
            self.attn = nn.MultiheadAttention(d, 2, dropout=drop, batch_f
irst=True)
            self.gate = nn.Parameter(torch.zeros(1))
        self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d, K))

    def forward(self, Z):
        z = self.proj(Z.float())
        if self.training and self.chan_drop > 0:
            z = z * (torch.rand(z.shape[0], z.shape[1], 1, device=z.devic
e) > self.chan_drop).float()
        if self.mix:
            h = self.norm(z + self.chan_emb)
            z = z + torch.tanh(self.gate) * self.attn(h, h, h, need_weigh
ts=False)[0]
        return self.out(z.flatten(1))
```

```
def train_v2(Ztr, ytr, Zte, yte, mix, seed, epochs=100, patience=20, bs=6
4):
    torch.manual_seed(seed)
    np.random.seed(seed)
    idx_tr, idx_va = train_test_split(np.arange(len(ytr)), test_size=0.2,
stratify=ytr, random_state=seed)
    Ztr_t, Zte_t = torch.tensor(Ztr, device=device), torch.tensor(Zte, de
vice=device)
    ytr_t = torch.tensor(ytr, dtype=torch.long, device=device)
    model = MixerV2(Ztr.shape[1], int(ytr.max()) + 1, mix).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.0
5)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)

    def predict(Z):
        model.eval()
        with torch.no_grad():
            return torch.cat([model(Z[i:i + 1024]).argmax(1) for i in ran
ge(0, len(Z), 1024)]).cpu().numpy()

    va_idx = torch.tensor(idx_va, device=device)
    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = torch.tensor(np.random.permutation(idx_tr), device=device)
        for i in range(0, len(perm), bs):
            b = perm[i:i + bs]
            loss = F.cross_entropy(model(Ztr_t[b]), ytr_t[b], label_smoot
hing=0.1)
            opt.zero_grad()
            loss.backward()
            opt.step()
        sched.step()
        va = np.mean(predict(Ztr_t[va_idx]) == ytr[idx_va])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.clone() for k, v in model.state_dict().ite
ms()}
        else:
            wait += 1
            if wait >= patience:
```

```
            break
    model.load_state_dict(best_state)
    gate = torch.tanh(model.gate).item() if mix else 0.0
    return float(np.mean(predict(Zte_t) == yte)), ep + 1, gate
```

In [7]:

```
# Run mixer v2 vs the same head without mixing, 4 small datasets, 3 seeds
each
RF = {'FingerMovements': 0.5500, 'Blink': 0.9956, 'UCI-HAR': 0.9013, 'Epi
lepsy-EEG': 0.9558, 'CAP-ID': 0.8091, 'CAP-OOD': 0.7858}
rows2 = []

def safe(fn):
    try:
        fn()
    except Exception as e:
        print('FAILED:', type(e).__name__, e, flush=True)

def run_v2(name, Ztr, ytr, Zte, yte, **kw):
    le = LabelEncoder().fit(ytr)
    ytr, yte = le.transform(ytr), le.transform(yte)
    for mix in (False, True):
        t0 = time.time()
        res = [train_v2(Ztr, ytr, Zte, yte, mix, s, **kw) for s in (0, 1,
2)]
        accs, gates = [r[0] for r in res], [round(r[2], 3) for r in res]
        method = 'mixer v2' if mix else 'concat + channel dropout'
        rows2.append(dict(dataset=name, method=method, mean=np.mean(acc
s), std=np.std(accs), rf=RF[name],
                          per_seed=json.dumps(accs), epochs=json.dumps([r
[1] for r in res]), gate=json.dumps(gates),
                          time_s=round(time.time() - t0)))
        print(f'{name:16s} {method:26s} {np.mean(accs):.4f} +/- {np.std(a
ccs):.4f} gate {gates} '
              f'{time.time() - t0:.0f}s', flush=True)
    pd.DataFrame(rows2).to_csv(f'{OUT}/mixer_v2_results.csv', index=Fals
e)

for name in ['FingerMovements', 'Blink']:
    d = load_ts_archive(name)
    safe(lambda: run_v2(name, fixed(d['train'][0]), d['train'][1], fixed
(d['test'][0]), d['test'][1]))
for folder, name in [('HAR', 'UCI-HAR'), ('Epilepsy', 'Epilepsy-EEG')]:
    d = load_tfc(folder)
    safe(lambda: run_v2(name, fixed(d['train'][0]), d['train'][1], fixed
(d['test'][0]), d['test'][1]))
```

```
FingerMovements  concat + channel dropout   0.5667 +/- 0.0047 gate [0.
0, 0.0, 0.0] 8s
FingerMovements  mixer v2                   0.5433 +/- 0.0411 gate [0.
001, 0.002, -0.059] 3s
Blink            concat + channel dropout   0.9844 +/- 0.0036 gate [0.
0, 0.0, 0.0] 4s
Blink            mixer v2                   0.9978 +/- 0.0018 gate [0.
051, -0.057, -0.057] 7s
UCI-HAR          concat + channel dropout   0.9102 +/- 0.0060 gate [0.
0, 0.0, 0.0] 26s
UCI-HAR          mixer v2                   0.9178 +/- 0.0024 gate [0.
14, 0.137, 0.141] 57s
Epilepsy-EEG     concat + channel dropout   0.9566 +/- 0.0010 gate [0.
0, 0.0, 0.0] 1s
Epilepsy-EEG     mixer v2                   0.9576 +/- 0.0011 gate [-
0.038, 0.038, -0.034] 1s
```

In [8]:

```
# CAP: 19 channels, about 40,000 samples - enough data for channel mixing
!pip install -q gdown h5py
import gdown, h5py
t0 = time.time()
gdown.download(id='1NFwX2CqLrenWF4az0c6J-OglAoD48PAT', output=f'{DATA}/ca
p.zip', quiet=True)
with zipfile.ZipFile(f'{DATA}/cap.zip') as z:
    z.extractall(f'{DATA}/CAP')
os.remove(f'{DATA}/cap.zip')
h5path = [os.path.join(r, f) for r, _, fs in os.walk(f'{DATA}/CAP') for f
in fs if f.endswith('.h5')][0]
print('CAP ready', f'{time.time() - t0:.0f}s', flush=True)
Zc, yc = {}, {}
with h5py.File(h5path, 'r') as h:
    for m in sorted(h.keys()):
        data, parts = h[f'{m}/data'], []
        for s in range(0, data.shape[0], 2000):
            parts.append(fixed(np.transpose(data[s:s + 2000], (0, 2,
1))))
        Zc[m], yc[m] = np.concatenate(parts), h[f'{m}/labels'][:].ravel()
        print(m, Zc[m].shape, f'{time.time() - t0:.0f}s', flush=True)
```

```
CAP ready 137s
Machine0 (12769, 19, 512) 184s
Machine1 (4694, 19, 512) 202s
Machine2 (4732, 19, 512) 220s
Machine3 (9930, 19, 512) 258s
Machine4 (8265, 19, 512) 290s
```

In [9]:

```
# CAP-ID (random split, seed 0, sizes as in the paper) and CAP-OOD (train
Machines 0, 1, 3; test Machine 4)
ms = sorted(Zc)
Z_all, y_all = np.concatenate([Zc[m] for m in ms]), np.concatenate([yc[m]
for m in ms])
n = len(y_all)
perm = np.random.RandomState(0).permutation(n)
tr, te = perm[:int(0.6375 * n)], perm[n - int(round(0.25 * n)):]
safe(lambda: run_v2('CAP-ID', Z_all[tr], y_all[tr], Z_all[te], y_all[te],
epochs=40, patience=8, bs=256))
trm = ['Machine0', 'Machine1', 'Machine3']
safe(lambda: run_v2('CAP-OOD', np.concatenate([Zc[m] for m in trm]), np.c
oncatenate([yc[m] for m in trm]),
                    Zc['Machine4'], yc['Machine4'], epochs=40, patience=
8, bs=256))
print(pd.DataFrame(rows2)[['dataset', 'method', 'mean', 'std', 'rf', 'gat
e']].round(4).to_string(index=False))
```

```
CAP-ID           concat + channel dropout   0.8166 +/- 0.0013 gate [0.
0, 0.0, 0.0] 16s
CAP-ID           mixer v2                   0.8258 +/- 0.0014 gate [0.
135, 0.144, -0.108] 29s
CAP-OOD          concat + channel dropout   0.7488 +/- 0.0009 gate [0.
0, 0.0, 0.0] 19s
CAP-OOD          mixer v2                   0.7679 +/- 0.0033 gate [0.
141, 0.146, -0.109] 30s
        dataset                    method   mean    std     rf
gate
FingerMovements concat + channel dropout 0.5667 0.0047 0.5500
[0.0, 0.0, 0.0]
FingerMovements                 mixer v2 0.5433 0.0411 0.5500  [0.001,
0.002, -0.059]
          Blink concat + channel dropout 0.9844 0.0036 0.9956
[0.0, 0.0, 0.0]
          Blink                 mixer v2 0.9978 0.0018 0.9956 [0.051,
-0.057, -0.057]
        UCI-HAR concat + channel dropout 0.9102 0.0060 0.9013
[0.0, 0.0, 0.0]
        UCI-HAR                 mixer v2 0.9178 0.0024 0.9013    [0.1
4, 0.137, 0.141]
   Epilepsy-EEG concat + channel dropout 0.9566 0.0010 0.9558
[0.0, 0.0, 0.0]
   Epilepsy-EEG                 mixer v2 0.9576 0.0011 0.9558 [-0.038,
0.038, -0.034]
         CAP-ID concat + channel dropout 0.8166 0.0013 0.8091
[0.0, 0.0, 0.0]
         CAP-ID                 mixer v2 0.8258 0.0014 0.8091  [0.135,
0.144, -0.108]
        CAP-OOD concat + channel dropout 0.7488 0.0009 0.7858
[0.0, 0.0, 0.0]
        CAP-OOD                 mixer v2 0.7679 0.0033 0.7858  [0.141,
0.146, -0.109]
```